Mini-ERP: Integration of System 1 (Catalogue) and System 2 (Sales/Order)

Integration specification — unified database design, reconciliation decisions, business rules, and handoff contract.

1. Purpose and Scope

This document merges the Product/Catalogue Management design (System 1) and the Sales/Order Management design (System 2) into one relational schema. It targets MySQL 8.0.16 or later (or MariaDB 10.2.1 or later), because older versions parse CHECK constraints but do not enforce them.

Included: Category, Product, Seller, SellerOffer (System 1); Customer, SalesOrder, OrderItem, OrderStatusHistory (System 2).

Not included yet: Inventory, Billing, Delivery. The Sales design references them, but their tables will be added after Systems 3 to 5 are reviewed.

2. Reconciliation Decisions

The two documents disagree in several places. These are the resolutions built into the unified design. Each must be confirmed by both groups.

#	Conflict	Resolution
1	System 1's SellerOffer uses the composite key (seller_id, product_id). System 2 expects a single offer_id.	Add a surrogate primary key `offer_id`. Keep (seller_id, product_id) as a UNIQUE constraint, so System 1's Rule 6 still holds.
2	System 1 calls the price `selling_price`. System 2 calls it `current_price`.	Use `selling_price`. System 2 maps it to its price snapshot.
3	The ERD image uses Seller.`status`. The System 1 document uses `seller_status`.	Use `seller_status`.
4	System 2 requires one seller per order but nothing in its schema enforces it.	Add `seller_id` to SalesOrder and OrderItem, with composite foreign keys, so the database itself rejects a mixed-seller order.

#	Conflict	Resolution
5	System 2 validates only offer_status. System 1 defines availability from product, offer and seller status together.	Orderable means all three are active. A view (`v_orderable_offer`) provides this in one place.
6	Status values are free text in System 2.	Use fixed values enforced by CHECK constraints (Section 4).
7	System 1 leaves open what happens to existing orders when an offer or product goes inactive.	Proposed: existing orders are unaffected because they hold price and name snapshots. Inactive status only blocks new orders. Needs class agreement.
8	Customer ownership is undecided in System 2.	Customer stays in Sales for now. It is isolated so it can be swapped for a shared account table later.

3. Entities and Relationships (ERD)

3.1 Entities

Entity	Owner	Primary key	Foreign keys
Category	System 1	category_id	none
Product	System 1	product_id	category_id → Category
Seller	System 1	seller_id	none
SellerOffer	System 1	offer_id	seller_id → Seller, product_id → Product
Customer	System 2	customer_id	none
SalesOrder	System 2	order_id	customer_id → Customer, seller_id → Seller

Entity	Owner	Primary key	Foreign keys
OrderItem	System 2	order_item_id	(order_id, seller_id) → SalesOrder, (offer_id, seller_id) → SellerOffer
OrderStatusHistory	System 2	history_id	order_id → SalesOrder

3.2 Relationships

Relationship	Cardinality	Rule
Category → Product	1 to many	A category has one or more products. Each product has exactly one category.
Product → SellerOffer	1 to many	Each offer refers to one product. A product should have at least one offer (application-level rule).
Seller → SellerOffer	1 to many	A seller may have zero offers. Each offer has one seller.
Seller → SalesOrder	1 to many	Each order is for exactly one seller (demo scope).
Customer → SalesOrder	1 to many	A customer may have zero or many orders.
SalesOrder → OrderItem	1 to many	A draft may be empty. A submitted order needs at least one item.
SellerOffer → OrderItem	1 to many	An offer may appear in many order items. Each item references one offer.
SalesOrder → OrderStatusHistory	1 to many	Every status change adds a history row.

The integration point between the two systems is `OrderItem.offer_id → SellerOffer.offer_id`. Sales never copies catalogue records. It only references the offer and snapshots the price and product name at submission.

4. Status Values

Field	Allowed values
product_status	active, discontinued

Field	Allowed values
seller_status	active, inactive
offer_status	active, inactive
order_status	draft, pending_stock, awaiting_payment, confirmed, fulfilling, completed, cancellation_pending, cancelled

Use lowercase snake_case values everywhere so the other groups can share the same strings.

5. Unified SQL Schema (MySQL)

```
-- Requires MySQL 8.0.16+ (or MariaDB 10.2.1+) so CHECK constraints are
enforced.
-- All tables use InnoDB (the default), which is required for foreign keys.
CREATE DATABASE IF NOT EXISTS mini_erp CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE mini_erp;

-- ===== SYSTEM 1: CATALOGUE =====
CREATE TABLE category (
  category_id   INT NOT NULL AUTO_INCREMENT,
  category_name VARCHAR(100) NOT NULL,
  description   TEXT,
  PRIMARY KEY (category_id),
  UNIQUE KEY uq_category_name (category_name)
);

CREATE TABLE product (
  product_id     INT NOT NULL AUTO_INCREMENT,
  category_id    INT NOT NULL,
  product_name   VARCHAR(150) NOT NULL,
  description    TEXT,
  product_status VARCHAR(20) NOT NULL DEFAULT 'active',
  PRIMARY KEY (product_id),
  UNIQUE KEY uq_product_name (product_name),
  CONSTRAINT fk_product_category FOREIGN KEY (category_id)
    REFERENCES category (category_id),
  CONSTRAINT chk_product_status CHECK (product_status IN
('active','discontinued'))
);
```

```
CREATE TABLE seller (
  seller_id     INT NOT NULL AUTO_INCREMENT,
  seller_name   VARCHAR(150) NOT NULL,
  contact_email VARCHAR(255) NOT NULL,
  seller_status VARCHAR(20) NOT NULL DEFAULT 'active',
  PRIMARY KEY (seller_id),
  UNIQUE KEY uq_seller_email (contact_email),
  CONSTRAINT chk_seller_status CHECK (seller_status IN ('active','inactive'))
);

CREATE TABLE seller_offer (
  offer_id      INT NOT NULL AUTO_INCREMENT,
  seller_id     INT NOT NULL,
  product_id    INT NOT NULL,
  selling_price DECIMAL(10,2) NOT NULL,
  offer_status  VARCHAR(20) NOT NULL DEFAULT 'active',
  PRIMARY KEY (offer_id),
  UNIQUE KEY uq_offer_seller_product (seller_id, product_id),
  UNIQUE KEY uq_offer_id_seller (offer_id, seller_id),
  CONSTRAINT fk_offer_seller  FOREIGN KEY (seller_id)  REFERENCES seller
(seller_id),
  CONSTRAINT fk_offer_product FOREIGN KEY (product_id) REFERENCES product
(product_id),
  CONSTRAINT chk_offer_price  CHECK (selling_price >= 0),
  CONSTRAINT chk_offer_status CHECK (offer_status IN ('active','inactive'))
);

-- ===== SYSTEM 2: SALES / ORDER =====
CREATE TABLE customer (
  customer_id    INT NOT NULL AUTO_INCREMENT,
  customer_name  VARCHAR(150) NOT NULL,
  email          VARCHAR(255),
  contact_number VARCHAR(30) NOT NULL,
  created_at     DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (customer_id)
);

CREATE TABLE sales_order (
  order_id             INT NOT NULL AUTO_INCREMENT,
  order_number         VARCHAR(30) NOT NULL,
  customer_id          INT NOT NULL,
  seller_id            INT NOT NULL,
  submission_reference VARCHAR(100),
  order_date           DATETIME,
  order_status         VARCHAR(30) NOT NULL DEFAULT 'draft',
  order_total          DECIMAL(12,2) NOT NULL DEFAULT 0,
```

```
  recipient_name       VARCHAR(150),
  recipient_contact    VARCHAR(30),
  requested_delivery_address TEXT,
  cancellation_reason  TEXT,
  created_at           DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at           DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  PRIMARY KEY (order_id),
  UNIQUE KEY uq_order_number (order_number),
  UNIQUE KEY uq_submission_reference (submission_reference),  -- many NULLs
allowed
  UNIQUE KEY uq_order_id_seller (order_id, seller_id),
  CONSTRAINT fk_order_customer FOREIGN KEY (customer_id) REFERENCES customer
(customer_id),
  CONSTRAINT fk_order_seller   FOREIGN KEY (seller_id)   REFERENCES seller
(seller_id),
  CONSTRAINT chk_order_status CHECK (order_status IN
('draft','pending_stock','awaiting_payment',

'confirmed','fulfilling','completed','cancellation_pending','cancelled')),
  CONSTRAINT chk_order_total CHECK (order_total >= 0),
  CONSTRAINT chk_submitted_complete CHECK (
    order_status IN ('draft','cancelled') OR (
      order_date IS NOT NULL AND recipient_name IS NOT NULL AND
      recipient_contact IS NOT NULL AND requested_delivery_address IS NOT
NULL))
);

CREATE TABLE order_item (
  order_item_id         INT NOT NULL AUTO_INCREMENT,
  order_id              INT NOT NULL,
  seller_id             INT NOT NULL,
  offer_id              INT NOT NULL,
  product_name_at_order VARCHAR(150) NOT NULL,
  quantity              INT NOT NULL,
  unit_price_at_order   DECIMAL(10,2) NOT NULL,
  line_total            DECIMAL(12,2) GENERATED ALWAYS AS (quantity *
unit_price_at_order) STORED,
  PRIMARY KEY (order_item_id),
  UNIQUE KEY uq_order_offer (order_id, offer_id),
  -- both FKs share seller_id, so an order can only hold offers of its own
seller
  CONSTRAINT fk_item_order FOREIGN KEY (order_id, seller_id)
    REFERENCES sales_order (order_id, seller_id),
  CONSTRAINT fk_item_offer FOREIGN KEY (offer_id, seller_id)
    REFERENCES seller_offer (offer_id, seller_id),
```

```
  CONSTRAINT chk_item_qty   CHECK (quantity > 0),
  CONSTRAINT chk_item_price CHECK (unit_price_at_order >= 0)
);

CREATE TABLE order_status_history (
  history_id             INT NOT NULL AUTO_INCREMENT,
  order_id               INT NOT NULL,
  previous_status        VARCHAR(30),
  new_status             VARCHAR(30) NOT NULL,
  changed_at             DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  change_source          VARCHAR(50) NOT NULL,
  actor_reference        VARCHAR(100),
  reason                 TEXT,
  source_event_reference VARCHAR(100),
  PRIMARY KEY (history_id),
  UNIQUE KEY uq_source_event (source_event_reference),  -- many NULLs allowed
  CONSTRAINT fk_history_order FOREIGN KEY (order_id) REFERENCES sales_order
(order_id)
);

-- ===== INTEGRATION VIEW =====
-- The single definition of "can this offer be ordered right now?"
CREATE OR REPLACE VIEW v_orderable_offer AS
SELECT o.offer_id, o.seller_id, o.product_id, p.product_name, o.selling_price
FROM seller_offer o
JOIN product p ON p.product_id = o.product_id
JOIN seller  s ON s.seller_id  = o.seller_id
WHERE o.offer_status   = 'active'
  AND p.product_status = 'active'
  AND s.seller_status  = 'active';
```

All foreign keys use the default `ON DELETE NO ACTION` behaviour, so referenced rows cannot be deleted. This supports the retention rule in both documents.

6. Integrated Business Rules

6.1 Rules carried over from System 1

ID	Rule	Enforcement
C-01	Category names are unique.	UNIQUE constraint
C-02	Product names are globally unique.	UNIQUE constraint

ID	Rule	Enforcement
C-03	Every product has exactly one existing category.	NOT NULL foreign key
C-04	Every product must have at least one offer.	Application logic (cannot be a plain FK)
C-05	A seller cannot have duplicate offers for the same product.	UNIQUE (seller_id, product_id)
C-06	Different sellers may price the same product differently.	Price lives on SellerOffer
C-07	A price update changes the existing offer row. It never creates a new one.	Application logic
C-08	Making a seller inactive also deactivates all of their offers.	Application logic or trigger
C-09	Records are deactivated through status, not deleted.	No ON DELETE CASCADE
C-10	selling_price is non-negative DECIMAL(10,2).	CHECK constraint
C-11	Product status and offer status are independent.	Separate columns
C-12	Stock quantity is not stored here.	Belongs to Inventory (System 3)

6.2 Rules carried over from System 2

ID	Rule	Enforcement
S-01	Every order has a unique order_id and order_number.	PK and UNIQUE
S-02	Every order references an existing customer.	NOT NULL foreign key
S-03	Submission requires complete recipient details and at least one item.	CHECK for recipient fields. At-least-one-item is application logic.
S-04	Quantities are positive whole numbers.	CHECK (quantity > 0)
S-05	Selecting the same offer again increases its quantity.	UNIQUE (order_id, offer_id) plus application logic

ID	Rule	Enforcement
S-06	Submitted order details are locked.	Application logic or trigger
S-07	Line total = quantity × unit price at order. Order total = sum of line totals.	Generated column for line total. Order total by application logic or trigger.
S-08	Repeated submissions return the existing order.	UNIQUE submission_reference
S-09	Repeated external events are applied only once.	UNIQUE source_event_reference
S-10	Every accepted status transition writes a history row.	Application logic
S-11	Submitted orders, items and history are never deleted.	No cascade deletes

6.3 New rules created by the integration

ID	Rule	Enforcement
I-01	**One seller per order.** Every item's seller must equal its order's seller.	Composite foreign keys on order_item
I-02	**Orderable means all three are active:** offer_status, product_status and seller_status.	v_orderable_offer, checked at add-to-draft and again at submission
I-03	**Price snapshot.** At submission, unit_price_at_order is copied from seller_offer.selling_price and product_name_at_order from product.product_name.	Application logic at submit
I-04	**Snapshots are immutable.** After submission, later catalogue changes never alter existing order items.	S-06 lock
I-05	**Price-change confirmation.** If the current selling_price differs from the price the customer saw in the draft, show the new total and require acceptance before submitting.	Application logic

ID	Rule	Enforcement
I-06	**Inactive after submission.** Deactivating an offer, product or seller does not cancel or alter submitted orders. It only blocks new ones.	Proposed, needs agreement (see 2.7)
I-07	**Sales is read-only on the catalogue.** Sales never inserts or updates Category, Product, Seller or SellerOffer.	MySQL GRANT permissions per group account
I-08	**Catalogue is unaware of orders.** The Catalogue group never writes to sales tables.	Database permissions
I-09	**Currency is PHP.** All amounts use two decimal places.	DECIMAL(10,2) and DECIMAL(12,2)

7. Integration Flow (Catalogue to Sales)

1. **Browsing.** Sales reads `v_orderable_offer` to show what customers can buy.
2. **Adding to a draft.** Sales inserts an order_item with the offer_id, seller_id, quantity, and the price the customer is seeing. If the offer is already in the draft, increase its quantity instead.
3. **Submitting.** In one transaction:
   - Re-check each item against `v_orderable_offer`. Reject inactive items and name them.
   - Compare each item's current selling_price to the draft price. If any differ, stop and ask the customer to accept.
   - Write the final unit_price_at_order and product_name_at_order, recalculate order_total, set order_status to pending_stock, and add a status-history row.
4. **After submission.** Sales no longer depends on the catalogue for that order. Everything it needs is in the snapshot.

8. Items Still Needing Class Agreement

Item	Needs agreement from
Surrogate offer_id replaces the composite key (2.1)	Catalogue and Sales

Item	Needs agreement from
Field names selling_price and seller_status (2.2, 2.3)	Catalogue and Sales
seller_id added to sales_order for one-seller-per-order (2.4)	Sales
Treatment of existing orders when offers or products go inactive (2.7)	Catalogue, Sales, Inventory, Billing, Delivery
Customer ownership and any shared account table (2.8)	All groups
Shared MySQL server: where it is hosted and which account each group uses	Whole class
Shared database versus API access between systems	Whole class

9. Next Steps

- Confirm Section 8 with the Catalogue and Sales groups.
- Send Systems 3 to 5 for review. Inventory will attach to `seller_offer` or `product` (the mapping is still open). Billing and Delivery will attach to `sales_order` and `order_item`.
- Add the remaining tables and re-check all identifiers, status values and monetary precision across the full schema.